PANDAS

Pandas: Exploring Data using Series, Exploring Data using DataFrames, Index objects, Re index, Drop Entry, Selecting Entries, Data Alignment, Rank and Sort

PANDAS

Pandas is a Python library used **for working with data sets.**

Pandas library provides **high performance, easy to use data structures, and analysis tools for the python programming language**. It is an **open source python library** which **provides high performance data manipulation and analysis tool using its powerful data structures.**

The name "Pandas" has a reference to **both "Panel Data", and "Python Data Analysis".**

PANDAS

Dataframe consist of two dimensions; the first dimension is the row and the second dimension is the column that is what we mean by two-dimensional and size-mutable.

each row represent a sample or a record, and each column represent a variable

PANDAS

- Pandas deals with dataframes

Name	Dimension	Description
Dataframe	2	• two-dimensional size-mutable • potentially heterogeneous tabular data structure with labeled axes (rows and columns)

PANDAS

What Can Pandas Do?

Pandas gives you answers about the data. Like:

Is there a correlation between two or more columns?
What is average value?
Max value?
Min value?

Pandas are also able to delete rows that are not relevant, or contains wrong values, like empty or NULL values. This is called cleaning the data.

PANDAS

For Checking Pandas Version

The version string is stored under __version__ attribute.

```
[19]: import pandas as pd

      print(pd.__version__)
```

```
1.5.1
```

PANDAS IN PRACTICAL APPLICATIONS

1. Data Cleaning and Preprocessing
2. EDA
3. Time Series Analysis(Indexing by Time)
4. Data Visualization with Matplotlib and Seaborn
5. DB Operations
6. Geospatial Data Analysis(GeoPandas)

Numpy Vs Pandas

Pandas is built on top of NumPy. Many Pandas operations internally use NumPy arrays for performance.

Feature	NumPy	Pandas
Data Structure	N-dimensional arrays (`ndarray`)	Tabular data (`DataFrame` and `Series`)
Purpose	Focuses on numerical computations and matrix operations	Focuses on data manipulation, cleaning, and analysis
Data Type	Homogeneous (all elements of an array must have the same type)	Heterogeneous (DataFrame columns can have different types)
Indexing	Integer-based indexing	Label-based and integer-based indexing
Ease of Use	Requires more effort for tabular data manipulation	High-level, easy-to-use tools for working with structured data
Functions	Limited to numerical and linear algebra operations	Includes SQL-like operations, groupby, and time-series capabilities

Numpy Vs Pandas

When to Use NumPy

Working with numerical data in arrays or matrices.
Performing high-performance mathematical computations.
Needing multi-dimensional arrays for scientific calculations.

When to Use Pandas

Working with structured/tabular data (e.g., rows and columns).
Cleaning, filtering, and reshaping datasets.
Performing operations like merging, joining, and grouping data.

PANDAS

Pandas Codebase?

import pandas
mydataset = { ‘cars’: [“BMW”, “Volvo”, “Ford”], ‘passings’: [3, 7, 2] }
myvar = pandas.DataFrame(mydataset)
print(myvar)

Pandas as pd

Pandas is usually imported under the pd alias.

alias: In Python alias are an alternate name for referring to the same thing.
Create an alias with the "as" keyword while importing:

Syntax : **import pandas as pd**
Now the Pandas package can be referred to as pd instead of pandas.

PANDAS

[8]:
```
import pandas as pd

a = [1, "banana", 3]
```

```
myvar = pd.Series(a)

print(myvar)
```

```
0         1
1    banana
2         3
dtype: object
```

PANDAS

[3]:
```
import pandas as pd

mydataset = {
  'cars': ["BMW", "Volvo", "Ford"],
  'passings': [3, 7, 2]
}

myvar = pd.DataFrame(mydataset)

print(myvar)
```

```
    cars  passings
0    BMW         3
1  Volvo         7
2   Ford         2
```

PANDAS

Pandas Series

What is a Series?

A Pandas Series is like a column in a table.

It is a one-dimensional array holding data of any type.

the **Series wraps both a sequence of values and a sequence of indices, which we can access with the values and index attributes**.

Example : Create a simple Pandas Series from a list - int, float, string

```
[5]: import pandas as pd

     a = ["A", "B", "C"]

     x = pd.Series(a)

     print(x)
```

```
0    A
1    B
2    C
dtype: object

The datatype of the elements in the Series is int64.
```

PANDAS

Based on the values present in the series, the datatype of the series is decided.

[6]:
```
import pandas as pd

a = [1.1, 2.2, 3.3]

myvar = pd.Series(a)

print(myvar)
```

```
0    1.1
1    2.2
2    3.3
dtype: float64
```

[7]:
```
import pandas as pd

a = ["apple", "banana", "orange"]

myvar = pd.Series(a)

print(myvar)
```

```
0     apple
1    banana
2    orange
dtype: object
```

Series as generalized NumPy array

the Series object is basically interchangeable with a one-dimensional NumPy array. The essential difference is the presence of the index: **while the NumPy array has an *implicitly defined* integer index used to access the values, the Pandas Series has an *explicitly defined* index associated with the values**.

This explicit index definition gives the Series object additional capabilities. For example**, the index need not be an integer, but can consist of values of any desired type**.

PANDAS

```
In[7]: data = pd.Series([0.25, 0.5, 0.75, 1.0],
                        index=['a', 'b', 'c', 'd'])
       data

Out[7]: a    0.25
        b    0.50
        c    0.75
        d    1.00
        dtype: float64
```

And the item access works as expected:

```
In[8]: data['b']

Out[8]: 0.5
```

PANDAS

Series as specialized dictionary

a Pandas Series a bit like a specialization of a Python dictionary. A dictionary is a structure that maps arbitrary keys to a set of arbitrary values, and a Series is a structure that maps typed keys to a set of typed values

A Pandas Series makes operations much more efficient than Python dictionaries for certain operations.

PANDAS

```
In[11]: population_dict = {'California': 38332521,
                           'Texas': 26448193,
                           'New York': 19651127,
                           'Florida': 19552860,
                           'Illinois': 12882135}
        population = pd.Series(population_dict)
        population

Out[11]: California    38332521
         Florida       19552860
         Illinois      12882135
         New York      19651127
         Texas         26448193
         dtype: int64
```

```
In[12]: population['California']

Out[12]: 38332521
```

```
In[13]: population['California':'Illinois']

Out[13]: California    38332521
         Florida       19552860
         Illinois      12882135
         dtype: int64
```

PANDAS

Labels

If nothing else is specified, the values are labeled with their index number.

First value has index 0, second value has index 1 etc.

This label can be used to access a specified value.

PANDAS

Example : Return the second value of the Series:

[9]:
```
import pandas as pd

a = [1, 2, 3, 4, 5, 6, 7]

myvar = pd.Series(a)

print(myvar[1])
```

```
2
```

[10]:
```
import pandas as pd

a = [1, 2, 3, 4, 5, 6, 7]

myvar = pd.Series(a)

print(myvar[2:4])
```

```
2    3
3    4
dtype: int64
```

PANDAS

Create you own labels

[2]:
```
import pandas as pd

a = [1, 2, 3]

myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar)
```

```
x    1
y    2
z    3
dtype: int64
```

When you have created labels, you can access an item by referring to the label.

PANDAS

Example : Return the value of “y”:

[3]:
```
import pandas as pd

a = [1, 2, 3]

myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar["y"])
```

```
2
```

PANDAS

Pandas DataFrames

In Python Pandas module, DataFrame is a very basic and important type.

If a Series is an analog of a one-dimensional array with flexible indices, a DataFrame is an analog of a two-dimensional array with both flexible row indices and flexible column names.

DataFrame as a generalized NumPy array

```
In[18]:
area_dict = {'California': 423967, 'Texas': 695662, 'New York': 141297,
             'Florida': 170312, 'Illinois': 149995}

area = pd.Series(area_dict)
area

Out[18]: California    423967
         Florida       170312
         Illinois      149995
         New York      141297
         Texas         695662
         dtype: int64
```

Now that we have this along with the `population` `Series` from before, we can use a dictionary to construct a single two-dimensional object containing this information:

```
In[19]: states = pd.DataFrame({'population': population,
                               'area': area})
        states

Out[19]:             area      population
         California  423967    38332521
         Florida     170312    19552860
         Illinois    149995    12882135
         New York    141297    19651127
         Texas       695662    26448193
```

PANDAS

To create a DataFrame from different sources of data or other Python datatypes, we can use "DataFrame”.

Syntax of DataFrame() class :
DataFrame(data=None, index=None, columns=None, dtype=None, copy=False)

Example Create an Empty DataFrame

To create an empty DataFrame, pass no arguments to pandas.DataFrame() class. In this example, we create an empty DataFrame and print it to the console output.

PANDAS

[13]:
```
import pandas as pd

df = pd.DataFrame()

print(df)
```

```
Empty DataFrame
Columns: []
Index: []
```

Example : Create a simple Pandas DataFrame

[14]:
```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

#load data into a DataFrame object:

df = pd.DataFrame(data)

print(df)
```

```
   calories  duration
0       420        50
1       380        40
2       390        45
```

PANDAS

Example Create a simple Pandas DataFrame with Lables - Index

[1]:
```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)
```

```
      calories  duration
day1       420        50
day2       380        40
day3       390        45
```

PANDAS

Create Pandas DataFrame from List of Lists

To create Pandas DataFrame from list of lists, you can pass this list of lists as data argument to "pandas.DataFrame()".
Each inner list inside the outer list is transformed to a row in resulting DataFrame.

Example : Create DataFrame from List of Lists

```
[15]: import pandas as pd

      #list of lists

      data = [['a1', 'b1', 'c1'],
              ['a2', 'b2', 'c2'],
              ['a3', 'b3', 'c3']]

      df = pd.DataFrame(data)
      print(df)
```

```
    0   1   2
0  a1  b1  c1
1  a2  b2  c2
2  a3  b3  c3
```

PANDAS

Example : Create DataFrame from List of Lists with Column Names & Index

[16]:
```
import pandas as pd

#list of lists

data = [['a1', 'b1', 'c1'],
        ['a2', 'b2', 'c2'],
        ['a3', 'b3', 'c3']]

columns = ['C1', 'C2', 'C3']
index = ['R1', 'R2', 'R3']


df = pd.DataFrame(data, index, columns)
print(df)
```

```
    C1  C2  C3
R1  a1  b1  c1
R2  a2  b2  c2
R3  a3  b3  c3
```

PANDAS

Example : Create DataFrame from List of Lists with Different List Lengths

```
[17]: import pandas as pd
```

```
#list of lists
data = [['a1', 'b1', 'c1', 'd1'],
        ['a2', 'b2', 'c2'],
        ['a3', 'b3', 'c3']]

df = pd.DataFrame(data)
print(df)
```

```
    0   1   2     3
0  a1  b1  c1    d1
1  a2  b2  c2  None
2  a3  b3  c3  None
```

PANDAS

Create Pandas DataFrame from Python Dictionary

You can create a DataFrame from Dictionary by passing a dictionary as the data argument to Data Dictionary.

Example : Create DataFrame from Dictionary

```
[18]: import pandas as pd

mydictionary = {'names': ['raju', 'ramu', 'ravi', 'akash'],
'physics': [68, 74, 77, 78],
'chemistry': [84, 56, 73, 69],
'algebra': [78, 88, 82, 87]}

#create dataframe using dictionary
df_marks = pd.DataFrame(mydictionary)

print(df_marks)
```

```
   names  physics  chemistry  algebra
0   raju       68         84       78
1   ramu       74         56       88
2   ravi       77         73       82
3  akash       78         69       87
```

PANDAS

Pandas Read CSV

A simple way to store big data sets is to use CSV files (comma separated files). CSV files contains plain text and is a well know format that can be read by everyone.

[23]:
```
import pandas as pd

#load dataframe from csv
df1 = pd.read_csv("pandas.csv")
```

```
#print dataframe
print(df1)
```

PANDAS

	Name	maths	physics	chemisry
0	a	11	21	31
1	b	12	22	32
2	c	13	23	32
3	d	14	24	34

Note : If you have a large DataFrame with many rows, Pandas will only return the first 5 rows, and the last 5 rows:

PANDAS

[24]:
```
import pandas as pd

#load dataframe from csv
df2 = pd.read_csv("pandas1.csv")

#print dataframe
print(df2)
```

```
     Duration  Pulse  Maxpulse  Calories
0          60    110       130     409.1
1          60    117       145     479.0
2          60    103       135     340.0
3          45    109       175     282.4
4          45    117       148     406.0
..        ...    ...       ...       ...
164        60    105       140     290.8
165        60    110       145     300.0
166        60    115       145     310.2
167        75    120       150     320.4
168        75    125       150     330.4

[169 rows x 4 columns]
```

PANDAS

to_string() Method :

to_string() is used to print the entire DataFrame.

```
import pandas as pd

#load dataframe from csv
df3 = pd.read_csv("pandas1.csv")

#print dataframe
print(df3.to_string())
```

```
     Duration  Pulse  Maxpulse  Calories
0          60    110       130     409.1
1          60    117       145     479.0
2          60    103       135     340.0
3          45    109       175     282.4
4          45    117       148     406.0
5          60    102       127     300.0
6          60    110       136     374.0
7          45    104       134     253.3
8          30    109       133     195.1
9          60     98       124     269.0
10         60    103       147     329.3
11         60    100       120     250.7
12         60    106       128     345.3
13         60    104       132     379.3
14         60     98       123     275.0
15         60     98       120     215.2
16         60    100       120     300.0
17         45     90       112       NaN
18         60    103       123     323.0
19         45     97       125     243.0
20         60    108       131     364.2
21         45    100       119     282.0
22         60    130       101     300.0
23         45    105       132     246.0
24         60    102       126     334.5
25         60    100       120     250.0
26         60     92       118     241.0
27         60    103       132       NaN
28         60    100       132     280.0
29         60    102       129     380.3
30         60     92       115     243.0
```

PANDAS

Null Values :

```
import pandas as pd

#load dataframe from csv
df = pd.read_csv("pandas123.csv")
df
```

	Name	maths	physics	chemisry
0	a	11	21.0	31.0
1	b	12	22.0	32.0
2	c	13	NaN	32.0
3	d	14	24.0	NaN
4	e	15	25.0	35.0
5	f	16	26.0	36.0
6	g	17	27.0	37.0
7	h	18	28.0	38.0
8	i	19	29.0	39.0
9	j	20	30.0	40.0

PANDAS

Shape Method :

```
df.shape
(10, 4)
```

Viewing Data :

To see how the data looks, we can use the head () method, which shows just the first five rows if we put a number as an argument to this method, this will be the number of the first rows that are listed.

PANDAS

df.head() Method :

```
df.head()
```

	Name	maths	physics	chemisry
0	a	11	21.0	31.0
1	b	12	22.0	32.0
2	c	13	NaN	32.0
3	d	14	24.0	NaN
4	e	15	25.0	35.0

```
df.head(3)
```

	Name	maths	physics	chemisry
0	a	11	21.0	31.0
1	b	12	22.0	32.0
2	c	13	NaN	32.0

```
df.head(7)
```

	Name	maths	physics	chemisry
0	a	11	21.0	31.0
1	b	12	22.0	32.0
2	c	13	NaN	32.0
3	d	14	24.0	NaN
4	e	15	25.0	35.0
5	f	16	26.0	36.0
6	g	17	27.0	37.0

PANDAS

tail() Method :

The tail() method, which returns the last five rows by default.

```
df.tail()
```

	Name	maths	physics	chemisry
5	f	16	26.0	36.0
6	g	17	27.0	37.0
7	h	18	28.0	38.0
8	i	19	29.0	39.0
9	j	20	30.0	40.0

```
df.tail(7)
```

	Name	maths	physics	chemisry
3	d	14	24.0	NaN
4	e	15	25.0	35.0
5	f	16	26.0	36.0
6	g	17	27.0	37.0
7	h	18	28.0	38.0
8	i	19	29.0	39.0
9	j	20	30.0	40.0

PANDAS

Names of the columns or the names of the indexes :

If we want to know the names of the columns or the names of the indexes, we can use the DataFrame attributes columns and index respectively.
The names of the columns or indexes can be changed by assigning a new list of the same length to these attributes.

```
df.columns
Index(['Name', 'maths', 'physics', 'chemisry'], dtype='object')
df.index
RangeIndex(start=0, stop=10, step=1)
```

PANDAS

The values of any DataFrame can be retrieved as a Python array by calling its values attribute.

```
df.values

array([['a', 11, 21.0, 31.0],
       ['b', 12, 22.0, 32.0],
       ['c', 13, nan, 32.0],
       ['d', 14, 24.0, nan],
       ['e', 15, 25.0, 35.0],
       ['f', 16, 26.0, 36.0],
       ['g', 17, 27.0, 37.0],
       ['h', 18, 28.0, 38.0],
       ['i', 19, 29.0, 39.0],
       ['j', 20, 30.0, 40.0]], dtype=object)
```

PANDAS

Info About the Data:

The DataFrames object has a method called info(), that gives you more information about the data set.

info() Method :

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Name      10 non-null     object
 1   maths     10 non-null     int64
 2   physics   9 non-null      float64
 3   chemisry  9 non-null      float64
dtypes: float64(2), int64(1), object(1)
memory usage: 448.0+ bytes
```

PANDAS

describe() Method :

If we just want quick statistical information on all the numeric columns in a data frame, we can use the function describe().
The result shows the count, the mean, the standard deviation, the minimum and maximum, and the percentiles, by default, the 25th, 50th, and 75th, for all the values in each column or series

```
df.describe()
```

	maths	physics	chemisry
count	10.00000	9.000000	9.000000
mean	15.50000	25.777778	35.555556
std	3.02765	3.073181	3.282953
min	11.00000	21.000000	31.000000
25%	13.25000	24.000000	32.000000
50%	15.50000	26.000000	36.000000
75%	17.75000	28.000000	38.000000
max	20.00000	30.000000	40.000000

Selecting Data

```
df
```

	Name	maths	physics	chemisry
0	a	11	21.0	31.0
1	b	12	22.0	32.0
2	c	13	NaN	32.0
3	d	14	24.0	NaN
4	e	15	25.0	35.0
5	f	16	26.0	36.0
6	g	17	27.0	37.0
7	h	18	28.0	38.0

Reindexing

An important method on pandas objects is reindex, which means to create a new object with the data *conformed* to a new index.

```
In [91]: obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=['d', 'b', 'a', 'c'])

In [92]: obj
Out[92]:
d    4.5
b    7.2
a   -5.3
c    3.6
dtype: float64
```

Calling reindex on this Series rearranges the data according to the new index, introducing missing values if any index values were not already present:

```
In [93]: obj2 = obj.reindex(['a', 'b', 'c', 'd', 'e'])

In [94]: obj2
Out[94]:
a   -5.3
b    7.2
c    3.6
d    4.5
e    NaN
dtype: float64
```

PANDAS

For ordered data like time series, it may be desirable to do some interpolation or filling of values when reindexing. The method option allows us to do this, using a method such as ffill, which forward-fills the values:

```
In [95]: obj3 = pd.Series(['blue', 'purple', 'yellow'], index=[0, 2, 4])

In [96]: obj3
Out[96]:
0      blue
2    purple
4    yellow
dtype: object

In [97]: obj3.reindex(range(6), method='ffill')
Out[97]:
0      blue
1      blue
2    purple
3    purple
4    yellow
5    yellow
dtype: object
```

Dropping Entries from an Axis

Dropping one or more entries from an axis is easy if you already have an index array or list without those entries.

drop method will return a new object with the indicated value or values deleted from an axis:

PANDAS

```
In [105]: obj = pd.Series(np.arange(5.), index=['a', 'b', 'c', 'd', 'e'])

In [106]: obj
Out[106]:
a    0.0
b    1.0
c    2.0
d    3.0
e    4.0
dtype: float64

In [107]: new_obj = obj.drop('c')

In [108]: new_obj
Out[108]:
a    0.0
b    1.0
d    3.0
e    4.0
dtype: float64

In [109]: obj.drop(['d', 'c'])
Out[109]:
a    0.0
b    1.0
e    4.0
dtype: float64
```

Sorting and Ranking

Sorting a dataset by some criterion is another important built-in operation. To sort lexicographically by row or column index, use the sort_index method, which returns a new, sorted object:

```
In [201]: obj = pd.Series(range(4), index=['d', 'a', 'b', 'c'])

In [202]: obj.sort_index()
Out[202]:
a    1
b    2
c    3
d    0
dtype: int64
```

With a DataFrame, you can sort by index on either axis:

```
In [203]: frame = pd.DataFrame(np.arange(8).reshape((2, 4)),
   .....:                      index=['three', 'one'],
   .....:                      columns=['d', 'a', 'b', 'c'])

In [204]: frame.sort_index()
```

PANDAS

```
Out[204]:
       d  a  b  c
one    4  5  6  7
three  0  1  2  3

In [205]: frame.sort_index(axis=1)
Out[205]:
       a  b  c  d
three  1  2  3  0
one    5  6  7  4
```

PANDAS

To sort a Series by its values, use its sort_values method:

```
In [207]: obj = pd.Series([4, 7, -3, 2])

In [208]: obj.sort_values()
Out[208]:
2   -3
3    2
0    4
1    7
dtype: int64
```

PANDAS

Ranking assigns ranks from one through the number of valid data points in an array. The rank methods for Series and DataFrame are the place to look; by default rank breaks ties by assigning each group the mean rank:

```
In [215]: obj = pd.Series([7, -5, 7, 4, 2, 0, 4])

In [216]: obj.rank()
Out[216]:
0    6.5
1    1.0
2    6.5
3    4.5
4    3.0
5    2.0
6    4.5
dtype: float64
```

PANDAS

Ranks can also be assigned according to the order in which they're observed in the data:

```
In [217]: obj.rank(method='first')
Out[217]:
0    6.0
1    1.0
2    7.0
3    4.0
4    3.0
5    2.0
6    5.0
dtype: float64
```

Here, instead of using the average rank 6.5 for the entries 0 and 2, they instead have been set to 6 and 7 because label 0 precedes label 2 in the data.
You can rank in descending order, too:

PANDAS

```
# Assign tie values the maximum rank in the group
In [218]: obj.rank(ascending=False, method='max')
Out[218]:
0    2.0
1    7.0
2    2.0
3    4.0
4    5.0
5    6.0
6    4.0
dtype: float64
```

PANDAS

DataFrame can compute ranks over the rows or the columns:

```
In [219]: frame = pd.DataFrame({'b': [4.3, 7, -3, 2], 'a': [0, 1, 0, 1],
   .....:                       'c': [-2, 5, 8, -2.5]})

In [220]: frame
Out[220]:
   a    b    c
0  0  4.3 -2.0
1  1  7.0  5.0
2  0 -3.0  8.0
3  1  2.0 -2.5

In [221]: frame.rank(axis='columns')
Out[221]:
     a    b    c
0  2.0  3.0  1.0
1  1.0  3.0  2.0
2  2.0  1.0  3.0
3  2.0  3.0  1.0
```

PANDAS

Slice Operator :

If we want to select a subset of rows from a DataFrame, we can do so by indicating a range of rows separated by : inside the square brackets.

This is commonly known as a slice of rows.

Next instruction returns the slice of rows from the 9th to the 13th position.

Note : that the slice does not use the index labels as references, but the position

PANDAS

```
df[1:3]
```

	Name	maths	physics	chemisry
1	b	12	22.0	32.0
2	c	13	NaN	32.0

```
df[2:6]
```

PANDAS

If we want to select a subset of columns and rows using the labels as our references instead of the positions, we can use loc indexing:
Next instruction will return all the rows between the indexes specified in the slice before the comma, and the columns specified as a list after the comma.

```
df["Name"]

0    a
1    b
2    c
3    d
4    e
5    f
6    g
7    h
8    i
9    j
Name: Name, dtype: object

df[["Name","maths"]]

  Name  maths
0    a     11
1    b     12
2    c     13
3    d     14
4    e     15
5    f     16
```

Pandas - Cleaning Data

Data Cleaning :

Data cleaning means fixing bad data in your data set. Bad data could be:

Empty cells

Data in wrong format

Wrong data

Duplicates

```
import pandas as pd

df = pd.read_csv("pandas123.csv")
df
```

	Name	maths	physics	chemisry
0	a	11	21.0	31.0
1	b	12	22.0	32.0
2	c	13	NaN	32.0
3	d	14	24.0	NaN
4	e	15	25.0	35.0
5	f	16	26.0	36.0
6	g	17	27.0	37.0
7	h	18	28.0	38.0
8	i	19	29.0	39.0
9	j	20	30.0	40.0

is null() method :

```
df.isnull().values.any()

True
```

How many null values

```
df.isnull().sum()

Name        0
maths       0
physics     1
```

PANDAS

Remove Rows :

One way to deal with empty cells is to remove rows that contain empty cells.

dropna() method :

the dropna() method returns a new DataFrame, and will not change the original.

```
x=df.dropna()
x
```

	Name	maths	physics	chemisry
0	a	11	21.0	31.0
1	b	12	22.0	32.0
4	e	15	25.0	35.0
5	f	16	26.0	36.0
6	g	17	27.0	37.0
7	h	18	28.0	38.0
8	i	19	29.0	39.0
9	j	20	30.0	40.0

Replace Empty Values :

Another way of dealing with empty cells is to insert a new value instead.

This way you do not have to delete entire rows just because of some empty cells. **fillna() method :**

The fillna() method allows us to replace empty cells with a value:

```
y=df.fillna(10)
y
```

	Name	maths	physics	chemisry
0	a	11	21.0	31.0
1	b	12	22.0	32.0
2	c	13	10.0	32.0
3	d	14	24.0	10.0
4	e	15	25.0	35.0
5	f	16	26.0	36.0
6	g	17	27.0	37.0
7	h	18	28.0	38.0
8	i	19	29.0	39.0
9	j	20	30.0	40.0

Replace Only For Specified Columns

The example above replaces all empty cells in the whole Data Frame.

To only replace empty values for one column, specify the column name for the DataFrame:

```
z=df["physics"].fillna(10)
z
```

```
0    21.0
1    22.0
2    10.0
3    24.0
4    25.0
5    26.0
6    27.0
7    28.0
8    29.0
9    30.0
Name: physics, dtype: float64
```

PANDAS

Discovering Duplicates :

Duplicate rows are rows that have bee

By taking a look at our test data set,

we can assume that row 11 and 12

are duplicates.

To discover duplicates, we can

use the duplicated() method.

The duplicated() method returns a

Boolean values for each row:

```
import pandas as pd

d1 = pd.read_csv("pandasduplicate.csv")
d1
```

	Name	maths	physics	chemisry
0	a	11	21	31
1	b	12	22	32
2	c	13	23	32
3	d	14	24	34
4	a	11	21	31
5	b	12	22	32

```
d1.duplicated()
```

```
0    False
1    False
2    False
3    False
4     True
5     True
dtype: bool
```